

Event-Grounded Sparse Autoencoders for Vision-Language-Action Policies

Xinchen Jin Aditya Chatterjee Pranav Kumar Rohan Paleja

Department of Computer Science, Purdue University
West Lafayette, IN 47907

{jin548, chatte59, kumar649, rpaleja}@purdue.edu

Abstract

Vision-Language-Action (VLA) policies translate language and visual inputs into robot actions, where their hidden representations directly shape closed-loop behavior. However, mechanistic interpretability tools from language and vision-language models do not transfer cleanly to VLAs: outputs are robot actions rather than human-readable tokens, and interventions can only be tested via expensive closed-loop rollouts. We propose an event-grounded interpretability pipeline that anchors SAE feature analysis to behavioral events rather than text contexts. End-effector keyframes are clustered within each task using visual, state, and temporal cues, linking SAE features to behaviorally salient events and, via optional VLM annotations, to semantic context. To our knowledge, our pipeline is among the first to ground SAE-based VLA analysis in closed-loop behavioral events. Across two simulation architectures and a real-robot study, event-grounded ranking yields the strongest causal effects on OpenVLA and transfers to the continuous action chunks of $\pi_{0.5}$. SAE is a sparse but imperfect intervention basis: usability varies with architecture and intervention site, and aggressive intervention reveals safety and interpretability limits. Overall, event-grounded SAE analysis emerges as a practical starting point for behavior-anchored VLA interpretability, motivating future work on SAE features beyond action-aligned coordinates, finer-grained closed-loop evaluation, and safe interventions for high-stakes VLA deployments. Code is available at <https://github.com/xc-j/Event-SAE>.

1 Introduction

Vision-Language-Action (VLA) models map language instructions and visual observations to robot actions and are increasingly deployed in closed-loop physical settings [1, 2, 3]. Because VLA outputs are actions that directly drive robot motion rather than the human-readable tokens emitted by language and vision-language models, hidden representations in a VLA are not only useful for predicting next actions but also directly shape what the robot does next: small internal changes can translate into different closed-loop trajectories, contact events, or failure modes. What these representations encode therefore matters both for interpretability research and for safely deploying VLA policies [4, 5].

Mechanistic interpretability has matured in language and vision-language models, where intermediate representations can be related to human-readable tokens through tools such as the logit lens or sparse autoencoder (SAE) feature decoding [6, 7]. These methods exploit two properties of language model outputs: (i) candidate features can be named by projecting internal directions into vocabulary space [8, 9] or by inspecting their high-activating text contexts [10, 11]; (ii) text-based activation examples enable scalable automated explanation and validation [12, 13]. Neither property transfers cleanly to VLAs. First, feature naming is harder: the logit lens does not transfer cleanly: VLA outputs live in action space, so projecting onto a language vocabulary tends to yield tokens with unreliable semantic meaning. Second, behavioral validation is harder. Unlike a language model, where the

output token gives a direct semantic readout, a VLA produces a semantic outcome only after many correct downstream steps. So a one-step intervention is hard to read, and a multi-step intervention would need every step to land precisely, which is too hard to do reliably. These constraints motivate grounding VLA feature analysis in recurring behavioral events from rollouts, rather than in projections to the language vocabulary. Recent work has begun to study the internal representations of VLA policies [14, 15, 16, 17, 18, 19]. In the concurrent work of Swann et al. [14], candidate SAE features are first surfaced from activation statistics and then labeled by manually aligning their activation traces with rollout video. Behavior thus enters only at the interpretation stage, where manual labeling does not scale and visual co-occurrence cannot establish a feature’s causal role. Our pipeline instead scores every alive SAE feature against external behavioral events, achieving full coverage of the SAE basis rather than only the features that stand out in SAE activation statistics.

We use SAEs because they decompose superposed activations [20] into a sparse basis whose features can be enumerated and scored individually [10, 21], and its training is unsupervised. To select features worth testing, our pipeline uses SAE-independent behavioral signals (Figure 1): we extract kinematic keyframes via Automatic Waypoint Extraction [22], cluster into events, score features against clusters, and validate features through closed-loop interventions. Extraction and scoring are automatic, so the pipeline scales beyond per-feature manual labeling. Our contributions and findings are:

- **An automated, event-grounded SAE pipeline for VLAs.** We extract kinematic keyframes from rollouts, cluster them into task-local events, and score every alive SAE feature against these events without per-feature manual labeling. The event anchors are independent of SAE activations; feature ranking then scores SAE activations conditioned on these external behavioral events.
- **Architecture- and site-dependent intervention behavior.** The same pipeline yields qualitatively different results: on OpenVLA, individual event-aligned features measurably drop closed-loop success; on $\pi_{0.5}$, the PaliGemma VLA backbone barely responds to single-feature edits while the action expert collapses under nearly any ranking. Intervention sites are architecture-specific and do not transfer directly.
- **Limits of binary closed-loop evaluation.** Across simulation and a real-robot study, single-feature interventions cannot cleanly isolate causal variables and binary success rates capture only coarse behavioral effects. We do not claim a full mechanistic account; the findings motivate finer-grained evaluation metrics and methods.

2 Background and Related Work

Mechanistic Interpretability of LLMs and VLMs. In LLMs, prior work has studied how information is stored and transformed across layers and components [23, 24], and used tools such as the logit lens to inspect how predictions evolve over depth by projecting intermediate activations through the unembedding matrix [6, 8, 9]. These methods rely on the discrete, human-readable output vocabulary, which makes intermediate representations relatively easy to inspect and validate. The same advantage largely carries over to VLMs, whose outputs remain linguistic, and similar analyses have begun to uncover multimodal circuits and representations [25, 26].

Sparse Autoencoders. A line of interpretability work from Anthropic identified superposition in neural networks [20], introduced sparse autoencoders (SAEs) as a means of recovering more interpretable latent features from activations [10], and later scaled this framework to large language models, showing that SAE features can capture abstract, multilingual, and safety-relevant concepts and support causal interventions [21]. A growing body of work has applied SAEs to interpret LLMs more broadly [11, 12, 27, 13], and similar ideas have recently been extended to VLMs [28, 29, 30]. In both settings, feature semantics can often be read off from textual outputs, and feature validation can draw on large, semantically annotated corpora.

Mechanistic Interpretability of VLAs. Applying these tools to VLAs raises new challenges, since outputs are actions and intervention quality must be assessed through closed-loop behavior. Prior work suggests that VLA behavior may rely on shortcut-like mechanisms rather than cleanly transferable semantic abstractions [31, 32], making perfectly disentangled SAE features an unrealistic target. Non-SAE approaches include FFN-direction analysis for physical quantities such as end-effector speed and height [15] and text-latent intervention for task redirection [16]. Several recent works apply SAEs to VLA policies: Khan et al. [17] use SAEs for steering without analyzing feature

interpretability, and Grant et al. [18] find that per-token SAE processing is essential for preserving VLA action fidelity. Buurmeijer et al. [19] take a complementary linear-probe approach to observe and steer VLA features. Most directly comparable to our pipeline, concurrent work by Swann et al. [14] trains SAEs on VLA residual streams and classifies features as general or memorized using a classifier trained on activation-derived statistics. These SAE-on-VLA works either label features from SAE-internal statistics or evaluate steering without external behavioral grounding. Our pipeline instead scores SAE features against SAE-independent behavioral events extracted from rollouts.

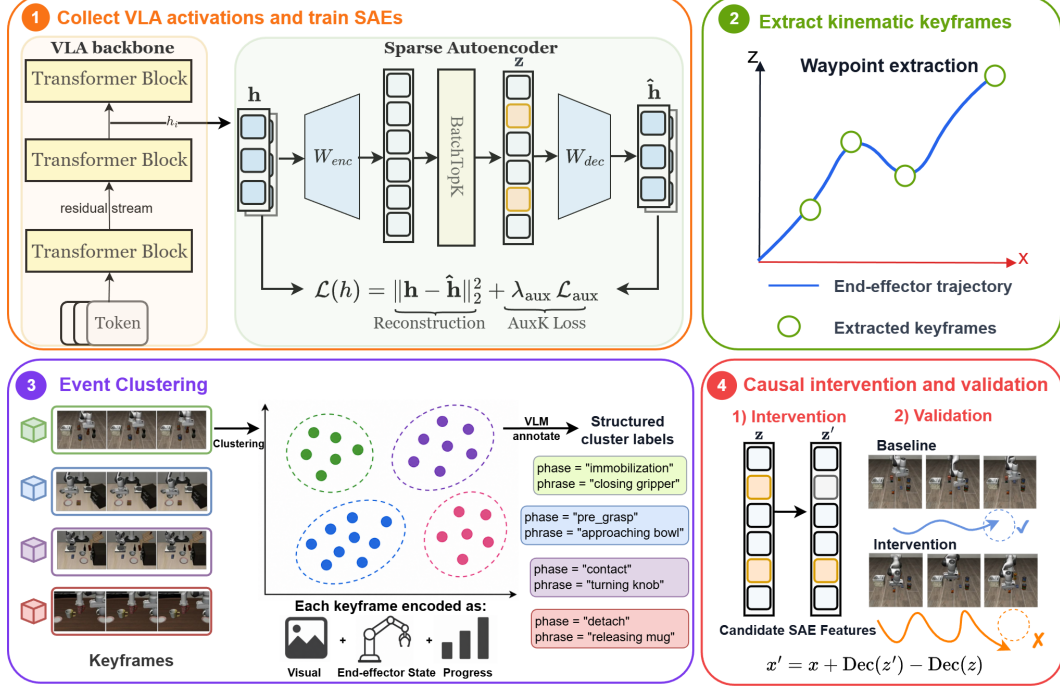


Figure 1: This figure shows the 4 stages of the event-grounded SAE pipeline: (1) SAE training (Sec. 3.1); (2) kinematic keyframe extraction (Sec. 3.2); (3) task-local event clustering with VLM annotation (Sec. 3.3); (4) closed-loop validation through residual-preserving latent edits (Sec. 3.5).

3 Method

Our pipeline turns closed-loop VLA rollouts into an SAE-based, event-grounded basis for causal feature analysis (Figure 1). Each upstream stage is anchored by a signal independent of the SAE activations being analyzed: candidate features are proposed from kinematic key moments that align with human intuition about a task’s salient steps. The remainder of this section covers SAE training (Section 3.1), kinematic keyframe extraction (Section 3.2), event clustering (Section 3.3), feature ranking (Section 3.4), and closed-loop intervention (Section 3.5), together giving a way to discover, score, and causally test SAE features without manual per-feature labeling.

3.1 SAE Training and Fidelity Evaluation

SAEs decompose dense hidden activations into a higher-dimensional sparse basis whose features are intended to be more interpretable than individual neurons [20, 10] (Stage (1) of Figure 1). Given a hidden state $h \in \mathbb{R}^d$, the encoder produces a sparse code $z = \text{BatchTopK}(W_{enc}h + b_{enc}) \in \mathbb{R}^m$ with $m \geq d$. The BatchTopK operator retains only the top kB pre-activations and zeros the rest, where B is the minibatch size and k is the average per-example active budget [33]. This architectural top- k constraint replaces the L_1 sparsity penalty of earlier SAEs. The decoder reconstructs as $\hat{h} = W_{dec}z + b_{dec}$, and we train both jointly to minimize $\mathcal{L}_{SAE} = \frac{1}{B} \sum_{i=1}^B \|h^{(i)} - \hat{h}^{(i)}\|_2^2 + \lambda_{aux} \mathcal{L}_{aux}$, where $\mathcal{L}_{aux}$ is the AuxK loss that revives dead features [7].

We train one BatchTopK SAE per (architecture stream, suite, layer) combination on per-token residual-stream activations from closed-loop rollouts; activations are not mean-pooled across tokens before training. SAE quality is assessed through two criteria: offline reconstruction diagnostics (FVE, alive fraction, average L_0), and behavioral fidelity under a reconstruction-only hook that replaces the hidden state at layer ℓ with $x' = \text{Dec}(\text{Enc}(x))$, reporting closed-loop success rate (SR); we call SR measured under this hook *Hooked SR*. Hooked SR selects the layer for downstream discovery and intervention.

3.2 Kinematic Keyframe Extraction

We use Automatic Waypoint Extraction (AWE) [22] to identify candidate event anchors (Stage (2) of Figure 1). AWE compresses a trajectory into a sparse ordered subsequence of waypoints under a reconstruction-error budget η , controlling the tradeoff between waypoint sparsity and trajectory fidelity. Concretely, AWE picks the smallest waypoint set such that linear interpolation between consecutive waypoints stays within η of the original trajectory at every timestep. Given a rollout, we apply AWE to the end-effector position trajectory and treat the resulting waypoints as kinematic keyframes, candidate temporal anchors for downstream event analysis. Because AWE depends only on end-effector motion, the resulting keyframes are independent of SAE activations, semantic labels, and feature scores, providing a representation-independent proposal mechanism for candidate events. For each keyframe at timestep t_i , we construct a local observation bundle: a five-frame image strip around t_i , a short video clip, and metadata including task description, episode index, success label, waypoint rank, and exact timestep. Figure 9 shows four representative bundles.

3.3 Event Clustering

To score SAE features against recurring event types rather than individual keyframes, we group keyframe bundles into task-local event clusters (Stage (3) of Figure 1). Clustering on end-effector position alone is insufficient: semantically distinct events can occur at spatially nearby positions, such as a grasp and a subsequent release when source and target locations are close, or two passes through the same region in opposite directions. We therefore form each cluster descriptor by concatenating a normalized visual embedding of the image strip (v), a normalized robot state vector with end-effector pose and gripper status (s), and a normalized temporal-progress scalar (p), with tunable weights controlling the relative contribution of each component (Appendix B).

We cluster descriptors task-locally with agglomerative clustering at a fixed cosine-distance threshold, keeping only clusters that recur in at least half of the task’s episodes; the retained clusters serve as event units for the feature ranking of Section 3.4. For OpenVLA+LIBERO, we additionally annotate each cluster with a short phrase and coarse phase label by querying a vision-language model (VLM) on exemplar keyframe bundles, with a phase taxonomy adapted from Chen et al. [34] (prompt template in Appendix C). These labels attach a semantic handle to each cluster, helping readers interpret which behavioral event (e.g., approach, contact, release) a feature responds to in Figure 9 and Appendix F. For illustration, Figure 2 visualizes the resulting clusters in end-effector position space for four $\pi_{0.5}$ LIBERO tasks; quantitative cluster statistics are reported in Section 4.2.

3.4 Feature Ranking Strategies

A trained SAE provides a sparse feature basis, but interpreting which of its many features are behaviorally meaningful is a separate problem [35]. Closed-loop intervention has a fixed budget per feature, so we need a ranking that selects a small candidate set worth testing. We compare four ranking strategies, each encoding a different hypothesis about what makes a feature important; Figure 3 illustrates the activation shape that each ranking scores highly. Per-episode score formulas for all four rankings are deferred to Appendix D.

Event-aligned ranking. For each event at timestep t_i , the event-aligned score measures how well a feature’s activation trace inside a $\pm w$ window matches one of three templates: a *pulse* spiking at t_i (e.g., a brief spike at the moment of contact), or a *step-up/step-down* marking a state transition (e.g., gripper opening or closing). The window mean is subtracted before matching, so the score reflects temporal shape rather than baseline activation level. Per-event scores use the best-matching template, are averaged within each episode, then averaged across episodes so each episode is weighted equally.

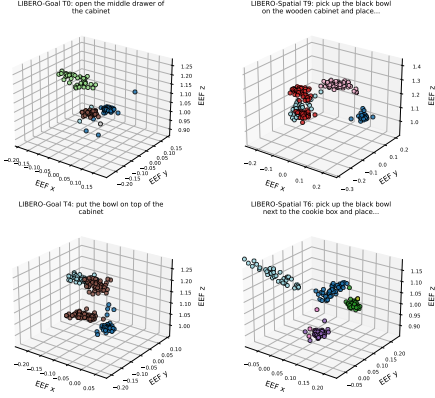


Figure 2: Example $\pi_{0.5}$ keyframe clusters in end-effector position space for four LIBERO tasks; each color is a task-local event cluster.

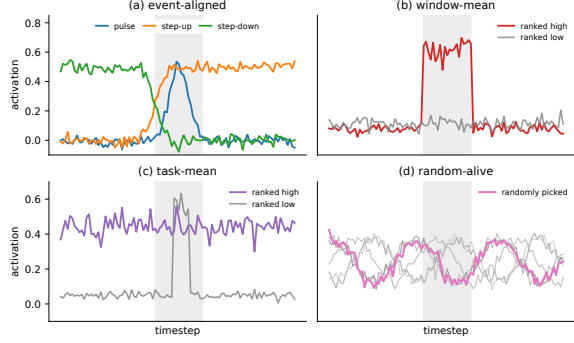


Figure 3: Schematic of the four ranking strategies; each panel shows the activation shape scored highly by the corresponding ranking, with the gray band marking the event window centered at t_i .

Window-mean ranking. The window-mean ranking shares event windows with the event-aligned ranking but ignores temporal structure within each window. It captures features with high average activation near events, without requiring the activation to follow a specific temporal pattern.

Task-mean ranking. The task-mean ranking ignores event windows entirely and ranks features by their average activation across all rollout timesteps in the suite. It tests whether features that are simply highly active during a task (regardless of when, or near which event) also tend to be causally important.

Random-alive ranking. As a sanity check, we also include a random ranking: we uniformly sample K features from the set of alive SAE features (those active in at least one rollout of the suite), excluding any feature already selected by the event-aligned, window-mean, or task-mean rankings. This exclusion ensures the random control measures the effect of generic alive features, not features that happen to overlap with informed rankings. The ranking serves as a lower bound: if the informed rankings did not outperform this control, they would carry no useful signal at all.

3.5 Intervention Modes

To test the causal role of selected SAE features, we intervene on hidden representations during inference and measure the effect on closed-loop behavior (Stage (4) of Figure 1). We use two intervention modes. The first is the reconstruction-only hook from Section 3.1, used to assess SAE fidelity. The second, described below, is a residual-preserving latent edit that modifies a chosen subset of features without disturbing the rest of the hidden state; the goal is to steer behavior rather than overwrite it completely.

Residual-preserving latent edits. Let $x \in \mathbb{R}^d$ be the hidden state at the intervention site and $z = \text{Enc}(x) \in \mathbb{R}^m$ its SAE code. We modify a target subset of latent features S by scaling them by a factor α :

$$z'_i = \begin{cases} \alpha z_i & i \in S, \\ z_i & i \notin S. \end{cases} \quad (1)$$

With this parameterization, $\alpha = 0$ zeroes the targeted features, $\alpha \in (0, 1)$ softly suppresses them, $\alpha = 1$ recovers z (no edit), and $\alpha > 1$ amplifies them. Rather than replacing x with the full perturbed reconstruction $\text{Dec}(z')$, we apply only the change induced by the edit: $x' = x + \text{Dec}(z') - \text{Dec}(z)$. Letting $\text{err}(x) = x - \text{Dec}(\text{Enc}(x))$ denote the SAE reconstruction error on x , this can be rewritten as $x' = \text{Dec}(z') + \text{err}(x)$, which makes explicit that the intervention modifies only the SAE-explained component of the hidden state while preserving the reconstruction error. This differs from a fixed decoder-vector addition $x' = x + \alpha \sum_{i \in S} d_i$ used in prior work [14], which is independent of the current sample’s latent values; we discuss this contrast further in Appendix K.

4 Experiments

4.1 Setup and SAE Fidelity

We evaluate the pipeline on OpenVLA [1] (discrete action tokens) and $\pi_{0.5}$ [3] (continuous action chunks via an action expert (AE) conditioned on a PaliGemma vision-language backbone (PG)) on the four LIBERO suites [36], with 50 closed-loop rollouts per task. We train one BatchTopK SAE [33] per (stream, suite, layer) at active budget $k=64$ on the post-block residual stream; SAE sizes appear in Table 1, full hyperparameters in Appendix A, and sanity checks on hook location and information retention in Appendix L. Following Section 3.1, Table 1 reports offline diagnostics (FVE, alive fraction, average L_0) and Hooked SR (10 rollouts per task) per (stream, layer); SR is the primary behavioral metric throughout.

Policy stream	Layer	SAE size	FVE $\uparrow$	Alive%	Avg L_0	Raw SR (%)	Hooked SR (%)
<i>OpenVLA residual</i>	0	4096 $\rightarrow$ 32768	0.911	94.1%	63.6	68.0 $\pm$ 14.3%	0.0 $\pm$ 0.0%
	16	4096 $\rightarrow$ 32768	0.999	3.1%	63.4		0.0 $\pm$ 0.0%
	24	4096 $\rightarrow$ 32768	0.998	3.4%	64.3		0.2 $\pm$ 0.5%
	31	4096 $\rightarrow$ 32768	0.995	8.0%	63.8		34.8 $\pm$ 24.5%
$\pi_{0.5}$ PG backbone	0	2048 $\rightarrow$ 2048	0.982	99.9%	63.2	96.8 $\pm$ 2.7%	96.5 $\pm$ 4.0%
	5	2048 $\rightarrow$ 2048	0.987	100.0%	63.0		96.5 $\pm$ 6.4%
	11	2048 $\rightarrow$ 2048	0.963	100.0%	63.0		97.8 $\pm$ 3.9%
	16	2048 $\rightarrow$ 2048	0.996	99.9%	64.3		95.8 $\pm$ 2.2%
$\pi_{0.5}$ action expert	0	1024 $\rightarrow$ 1024	0.997	40.6%	64.0	96.4 $\pm$ 2.6%	96.5 $\pm$ 2.5%
	5	1024 $\rightarrow$ 1024	0.997	14.8%	64.0		95.8 $\pm$ 3.9%
	11	1024 $\rightarrow$ 1024	0.984	84.9%	64.0		95.8 $\pm$ 2.2%
	17	1024 $\rightarrow$ 1024	0.959	96.7%	63.9		97.2 $\pm$ 2.9%

Table 1: This table reports suite-averaged offline SAE diagnostics and closed-loop fidelity for each policy stream and layer, with means $\pm$ standard deviations taken across the four LIBERO suites. Raw SR is the no-hook policy success rate; Hooked SR is the success rate when the SAE reconstruction replaces the hidden state at the corresponding layer. PG layer 17 is omitted because the final PG prefix output is discarded after KV-cache construction.

SAE quality and fidelity differ sharply across VLA architectures. Alive fractions vary widely across model and layer (Table 1): OpenVLA drops from 94% at layer 0 to 3–8% at deeper layers, $\pi_{0.5}$ PG saturates near 100% throughout, and AE ranges from 14% to 97%. Despite this heterogeneity, all SAEs reconstruct well offline (FVE ≥ 0.91 , mostly ≥ 0.96). Reconstruction quality, however, does not translate to closed-loop behavior: only OpenVLA layer 31 retains nonzero closed-loop SR across suites (34.8% Hooked SR vs. $\sim 70\%$ baseline), while $\pi_{0.5}$ sustains $\geq 95\%$ Hooked SR at every probed PG and AE layer. Neither FVE nor alive fraction alone predicts Hooked SR, motivating it as the primary fidelity metric.

Why early-layer OpenVLA hooks fail. OpenVLA decodes each action as seven tokens over 256 quantization bins, so small reconstruction errors at early layers can shift selected bins and compound across closed-loop steps; failed rollouts typically collapse to near-zero actions and stay there, since the policy receives no observation change to drive it out of this self-reinforcing stall. $\pi_{0.5}$ avoids this failure: its continuous flow-matching action chunks are more robust to small reconstruction perturbations than discrete-token outputs, consistent with Grant et al. [18]’s observations on action-token structure. Layer 31 is closest to the OpenVLA action head, so a hook there leaves the fewest downstream layers to amplify its reconstruction error; we use it for all subsequent OpenVLA discovery and intervention experiments.

4.2 Keyframe Extraction and Event Clustering

Applying AWE and task-local clustering (Sections 3.2, 3.3) to closed-loop rollouts yields the keyframe and event-cluster statistics in Table 2. Most raw event types survive the 50% episode-coverage filter as recurring clusters, and counts are comparable across the two policies on each suite, indicating similar end-effector kinematics despite the different action interfaces.

Recurring clusters span manipulation phases (approach, contact/grasp, transport, release, withdrawal) and feed all suite-level rankings in Section 3.4. VLM-assigned phrase and phase labels make each cluster human-readable: Figure 9 shows per-cluster keyframe bundles, and Figure 8 overlays the

Source	Suite	Keyframes/rollout	Clusters	Recurring
OpenVLA	LIBERO-Spatial	4.15	48	36
OpenVLA	LIBERO-Object	4.25	58	39
OpenVLA	LIBERO-Goal	3.86	50	37
OpenVLA	LIBERO-10	6.14	74	61
$\pi_{0.5}$	LIBERO-Spatial	4.11	46	36
$\pi_{0.5}$	LIBERO-Object	4.75	47	45
$\pi_{0.5}$	LIBERO-Goal	3.88	44	35
$\pi_{0.5}$	LIBERO-10	6.91	63	56

Table 2: Keyframe and event-cluster statistics. *Keyframes/rollout*: AWE-proposed kinematic keyframes per rollout. *Clusters*: total task-local event clusters per suite. *Recurring*: subset passing the 50% episode-coverage filter, feeding suite-level rankings (§3.4).

phase labels onto the rows of the event-feature score heatmaps. Together these views tell whether a high-scoring feature targets one specific event or recurs across phases, helping human analysis.

4.3 Closed-Loop Causal Intervention and Validation

Closed-loop intervention experiments stress-test which SAE features actually drive policy behavior. We focus on *single-feature* interventions throughout this section, since multi-feature edits entangle effects across features and obscure attribution; a joint multi-feature dropout is deferred to Appendix H. We compare two intervention modes: *hard* zero-out ($\alpha=0$), which silences a feature entirely and provides a clean on/off readout of feature causality; and *soft* intervention ($\alpha \in [0, 1]$), which sweeps edit strength continuously and is motivated by the saturation we observe under hard zero-out. We begin with hard zero-out, the strongest single-feature intervention available under our protocol. For each (policy, layer, suite), we compare four feature ranking strategies from Section 3.4: EVENT-ALIGNED, WINDOW-MEAN, TASK-MEAN, and a RANDOM-ALIVE control that uniformly samples from alive features after excluding all top selections of the other three rankings. For each ranking we take the top K features ($K=5$ for OpenVLA, $K=3$ for $\pi_{0.5}$) and apply the residual-preserving latent edit of Section 3.5 to each feature separately at edit strength $\alpha = 0$. We report $\Delta\text{SR} = \text{SR}_{\alpha=0} - \text{SR}_{\text{baseline}}$ (negative values indicate a drop) averaged across the K features under the rollout budget of Section 4.1. A ranking is behaviorally informative when its top features produce more negative ΔSR than the RANDOM-ALIVE control.

Policy stream	Layer	Baseline SR (%)	Zero-out SR (%) with ΔSR in parentheses			
			Event-aligned	Window-mean	Task-mean	Random-alive
OpenVLA residual	31	70.0%	48.8% (-21.2)	63.8% (-6.2)	63.5% (-6.5)	68.7% (-1.3)
$\pi_{0.5}$ PG backbone	0	96.8%	94.7% (-2.2)	95.5% (-1.4)	96.0% (-0.9)	96.2% (-0.7)
	5	96.8%	96.0% (-0.9)	95.8% (-1.0)	96.7% (-0.2)	96.3% (-0.5)
	11	96.8%	94.2% (-2.7)	94.7% (-2.2)	94.0% (-2.8)	95.8% (-1.0)
	16	96.7%	96.0% (-0.7)	96.7% (+0.0)	95.2% (-1.5)	96.3% (-0.3)
$\pi_{0.5}$ action expert	0	96.4%	0.0% (-96.4)	0.0% (-96.4)	0.0% (-96.4)	95.7% (-0.7)
	5	96.4%	0.8% (-95.6)	0.2% (-96.2)	0.2% (-96.2)	73.0% (-23.4)
	11	96.4%	0.0% (-96.4)	0.2% (-96.2)	0.0% (-96.4)	88.3% (-8.1)
	17	96.4%	15.2% (-81.2)	0.0% (-96.4)	0.0% (-96.4)	89.3% (-7.1)

Table 3: Closed-loop SR after zeroing selected SAE features. Mean SR (%) is followed in parentheses by $\Delta\text{SR} = \text{SR}_{\alpha=0} - \text{SR}_{\text{baseline}}$ in percentage points (negative indicates lower success). Each entry averages over the top K features per ranking.

Cross-architecture results. Table 3 shows three distinct intervention regimes. For OpenVLA layer 31, the event-aligned ranking produces the largest SR drop, well separated from the random-alive control, while window-mean and task-mean rankings fall in between; temporal event alignment thus adds useful information beyond feature activation magnitude alone. Within the OpenVLA event-aligned ranking, the top three features carry nearly all of the SR drop (per-feature breakdown in Appendix I), so a small set of high-weight features dominates the closed-loop signal. A target/off-

target probe on event-cluster-selected features finds nearly identical SR drops (Appendix J), suggesting many top-ranked features participate in shared manipulation computations across tasks within a suite.

The $\pi_{0.5}$ PaliGemma backbone stream behaves very differently: across all probed layers and ranking strategies, single-feature edits leave SR close to baseline, suggesting limited direct leverage under the residual-preserving zero-out protocol. By contrast, the action-expert stream collapses under the same protocol, with top-ranked AE features driving SR to near zero across most (layer, ranking) combinations. AE rankings are therefore poorly distinguished by zero-out alone: window-mean, task-mean, and even random-alive features all produce comparable disruption. We read the AE results as evidence of broad causal sensitivity rather than event-specific selectivity, matching the PG/AE asymmetry reported by Swann et al. [14]. Neither $\pi_{0.5}$ stream shows comparable top- K concentration: PG drops stay flat across ranks, and AE drops are uniformly deep across top-1/2/3. This is consistent with the architectural roles. The PaliGemma backbone influences actions only indirectly: the action expert reads its hidden state through the cross-attention KV-cache, which mixes each prefix token with all others and dilutes any single-feature edit before it reaches the action output. AE features instead modulate the residual stream that directly produces the action chunk, with no such bottleneck. Consistent with this, jointly dropping 16 PG features still yields only moderate SR change (Appendix H): PG needs far heavier perturbation than AE to move SR. Qualitative failure examples (Appendix G) show that the SR drops correspond to visible closed-loop degradation rather than only minor trajectory deviations.

Setting	Event $\cap$ Window	Event $\cap$ Task	Window $\cap$ Task
$\pi_{0.5}$ all	48.0%	48.0%	97.0%
$\pi_{0.5}$ AE	43.7%	43.7%	93.7%
$\pi_{0.5}$ PG	52.0%	52.0%	100.0%
OpenVLA layer 31	5.0%	5.0%	95.0%

Table 4: This table reports average top- K feature overlap as a percentage of K between ranking strategies. RANDOM-ALIVE overlap with the other three rankings is zero by construction. Window-mean and task-mean nearly coincide; event-aligned selects largely distinct features.

Soft intervention: α_f as a controllable dial. The hard zero-out failures and the AE saturation in Table 3 both arise from a single hyperparameter: the edit strength $\alpha = 0$. To trace how behavior changes with intervention magnitude, we sweep α_f continuously between 0 (full zero-out) and 1 (identity). Figure 4 shows that for $\pi_{0.5}$ AE on LIBERO-10, SR transitions monotonically between collapse and baseline at every probed layer, so the magnitude of behavioral disruption can be selected directly rather than only chosen between identity and policy collapse. The transition is steep, however: SR rises rapidly over a narrow band of α_f , so binary SR responds more like a switch than a gradual dial. This makes real-robot tuning sensitive to small mis-calibrations: a slightly too-aggressive α_f can push the policy into the arm-instability failure mode visualized in Figure 10.

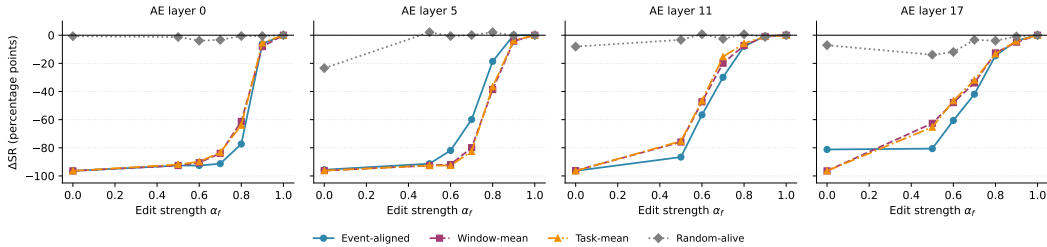


Figure 4: Simulator dose-response on LIBERO-10: $\pi_{0.5}$ action-expert per-layer Δ SR (percentage points) as α_f sweeps from 0 (zero-out) to 1 (identity), one line per ranking.

Case study: ranking overlap. We also characterize what each ranking selects (Table 4). Window-mean and task-mean rankings nearly coincide: features highly active in event windows tend to be highly active throughout the task. Event-aligned rankings, in contrast, select largely distinct features, showing that temporal change does not require high baseline magnitude. The same ranking strategies also select different feature sets across architectures: event-aligned and magnitude rankings are almost disjoint on OpenVLA layer 31 but overlap by $\sim 50\%$ on $\pi_{0.5}$.

4.4 Real-Robot Demonstration and Steering Safety

Setup. A single-arm Mobile ALOHA platform with a wrist-mounted and a third-person camera runs a LoRA-finetuned $\pi_{0.5}$ [3] on a chip-approach task. Each rollout is conditioned on either “Approach the red chips” or “Approach the yellow chips” with counterbalanced layouts (Figure 6), so successful behavior requires visual localization rather than trajectory replay. We analyze action-expert SAE features as in Section 4.1; hardware details and a baseline reproduction of Häon et al. [15]’s FFN value-vector projection are in Appendix E.

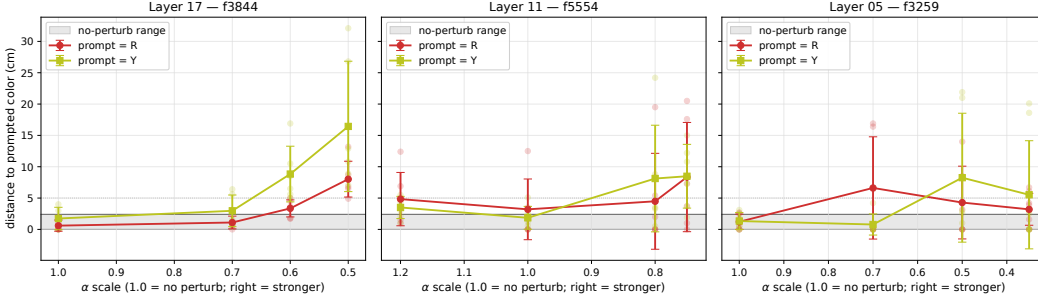


Figure 5: Real hardware dose-response: end-effector distance to the prompted chip cluster vs SAE-suppression strength at three AE layers. Lower distance means the rollout follows the prompt more faithfully; the shaded band is the no-perturbation range.

Steering effects depend on layer and strength. For all hardware trials, we use conservative latent-space suppression because aggressive or poorly tuned interventions can collapse behavior in simulation and induce large real-robot motions, posing risks to the robot and nearby personnel (Appendix K). Figure 5 shows that SAE suppression measurably alters color-conditioned approach behavior, with the strongest effect at action-expert layer 17: stronger suppression produces a larger deviation from the prompted chip cluster. Earlier layers are noisier. Selected SAE coordinates can therefore affect real-robot prompt-following, with effect magnitude depending on layer, feature, and suppression strength. The hardware result supports conservative, safety-aware steering as the deployable form of SAE-based VLA intervention. More broadly, hard zero-out probes causal sensitivity for ranked SAE coordinates, not clean factor-level isolation: SAE features may be polysemantic and large edits can move the policy off-distribution.

5 Limitations and Conclusion

Limitations. Three caveats apply. (i) Shared, not task-specific features: a feature selected for one task disrupts other tasks about equally (target/off-target probe), and on the action expert even random features collapse behavior. Top-ranked features therefore do work shared across tasks, so a single-feature edit cannot isolate one behavioral factor. (ii) Underused VLM semantics: VLM-assigned phrase and phase labels serve only as visualization aids on the event-feature heatmaps and cluster bundles (Appendix F), not as part of ranking or intervention scoring, leaving the cluster-level semantic structure largely unexploited. (iii) Coarse, safety-limited evaluation: binary success rate measures task success, not how selectively an edit changes behavior; strong edits also destabilize the real robot, so only mild interventions were safe to test.

Conclusion. We introduce event-grounded SAE analysis as a behavior-anchored approach to VLA mechanistic interpretability. By linking kinematic keyframes, task-local event clusters, and event-aligned ranking, the pipeline scores every alive SAE feature without per-feature labeling and selects candidates for closed-loop causal testing. Across OpenVLA and $\pi_{0.5}$ in simulation, and on a real-robot LoRA-finetuned $\pi_{0.5}$, it reveals architecture- and site-dependent intervention behavior: localized event-aligned effects in OpenVLA, weak single-feature leverage in the $\pi_{0.5}$ PaliGemma backbone, and broad sensitivity in the action expert. SAE features thus provide a sparse but imperfect causal basis for VLA policies. Future work should move VLM-derived event semantics from visualization into ranking and intervention, replace binary success with finer-grained behavioral metrics, and develop safety-calibrated interventions for physical deployment. Together, these steps move toward more transparent and controllable VLA policies for real-world robotics.

References

- [1] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Open-VLA: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024. URL <https://openreview.net/forum?id=ZMnD6QZAE6>.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [3] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [4] Rohan Paleja, Yaru Niu, Andrew Silva, Chace Ritchie, Sugju Choi, and Matthew Gombolay. Learning interpretable, high-performing policies for autonomous driving. In *Robotics: Science and Systems (RSS)*, 2022.
- [5] Rohan Paleja, Muyleng Ghuy, Nadun Ranawaka Arachchige, Reed Jensen, and Matthew Gombolay. The utility of explainable ai in ad hoc human-machine teaming. *Advances in neural information processing systems*, 34:610–623, 2021.
- [6] nostalgebraist. Interpreting GPT: The logit lens, 2020. URL <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>. LessWrong blog post.
- [7] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=tcsZt9ZNKD>.
- [8] Mor Geva, Avi Caciularu, Kevin Wang, and Yoav Goldberg. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 30–45, 2022.
- [9] Guy Dar, Mor Geva, Ankit Gupta, and Jonathan Berant. Analyzing transformers in embedding space. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16124–16170, 2023.
- [10] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2(5):6, 2023.
- [11] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.
- [12] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models. <https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html>, 2023.
- [13] Gonalo Santos Paulo, Alex Troy Mallen, Caden Juang, and Nora Belrose. Automatically interpreting millions of features in large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=EemtbtJ0Xc>.
- [14] Aiden Swann, Lachlain McGranahan, Hugo Buurmeijer, Monroe Kennedy III, and Mac Schwager. Sparse autoencoders reveal interpretable and steerable features in vla models. *arXiv preprint arXiv:2603.19183*, 2026.

- [15] Bear Häon, Kaylene Stocking, Ian Chuang, and Claire Tomlin. Mechanistic interpretability for steering vision-language-action models. *arXiv preprint arXiv:2509.00328*, 2025.
- [16] Quanyi Li. Task reconstruction and extrapolation for π_0 using text latent. *arXiv preprint arXiv:2505.03500*, 2025.
- [17] Momin Ahmad Khan, Novak Boskov, Fatima M. Anwar, and Manzoor A. Khan. Controlling vision-language-action policies through sparse latent directions. In *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=wtf3ww1EOL>.
- [18] Bryce Grant, Xijia Zhao, and Peng Wang. Not all features are created equal: A mechanistic study of vision-language-action models. *arXiv preprint arXiv:2603.19233*, 2026.
- [19] Hugo Buurmeijer, Carmen Amo Alonso, Aiden Swann, and Marco Pavone. Observing and controlling features in vision-language-action models. *arXiv preprint arXiv:2603.05487*, 2026.
- [20] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.
- [21] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- [22] Lucy Xiaoyang Shi, Archit Sharma, Tony Z. Zhao, and Chelsea Finn. Waypoint-based imitation learning for robotic manipulation. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=X0cmlTh1Vl>.
- [23] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.
- [24] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12216–12235, 2023.
- [25] Clement Neo, Luke Ong, Philip Torr, Mor Geva, David Krueger, and Fazl Barez. Towards interpreting visual information processing in vision-language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chanJGoa7f>.
- [26] Anirudh Phukan, Divyansh Divyansh, Harshit Kumar Morj, Vaishnavi Vaishnavi, Apoorv Saxena, and Koustava Goswami. Beyond logit lens: Contextual embeddings for robust hallucination detection & grounding in vlms. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 9661–9675, 2025.
- [27] David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, and Joseph Isaac Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *Interpretable AI: Past, Present and Future*, 2024. URL <https://openreview.net/forum?id=Wzav8fesTL>.
- [28] Hantao Lou, Changye Li, Jiaming Ji, and Yaodong Yang. SAE-v: Interpreting multimodal models for enhanced alignment. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=S4HPn5Bo6k>.
- [29] Mateusz Pach, Shyamgopal Karthik, Quentin Bouniot, Serge Belongie, and Zeynep Akata. Sparse autoencoders learn monosemantic features in vision-language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=DaNnkQJSQf>.

- [30] Isabel Papadimitriou, Huangyuan Su, Thomas Fel, Sham M. Kakade, and Stephanie Gil. Interpreting the linear structure of vision-language model embedding spaces. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=qPsmGjqp1j>.
- [31] Chengyang He, Xu Liu, Gadiel Mark Sznaier Camps, Joseph Bruno, Guillaume Adrien Sartoretti, and Mac Schwager. Demystifying robot diffusion policies: Action memorization and a simple lookup table alternative. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=PL0tJ0fm7I>.
- [32] Yu Fang, Yuchun Feng, Dong Jing, Jiaqi Liu, Yue Yang, Zhenyu Wei, Daniel Szafir, and Mingyu Ding. When vision overrides language: Evaluating and mitigating counterfactual failures in vlas. *arXiv preprint arXiv:2602.17659*, 2026.
- [33] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders. In *NeurIPS 2024 Workshop on Scientific Methods for Understanding Deep Learning*, 2024. URL <https://openreview.net/forum?id=d4dp0CqybL>.
- [34] Kaiyuan Chen, Shuangyu Xie, Zehan Ma, Pannag R Sanketi, and Ken Goldberg. Robo2vlm: Improving visual question answering using large-scale robot manipulation data. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [35] Dong Shu, Xuansheng Wu, Haiyan Zhao, Daking Rai, Ziyu Yao, Ninghao Liu, and Mengnan Du. A survey on sparse autoencoders: Interpreting the internal mechanisms of large language models. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 1690–1712, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.89. URL <https://aclanthology.org/2025.findings-emnlp.89/>.
- [36] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [37] Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. Mobile ALOHA: Learning bimanual mobile manipulation using low-cost whole-body teleoperation. In *8th Annual Conference on Robot Learning*, 2024. URL <https://openreview.net/forum?id=F06tePGRZj>.

A SAE Training Details

We train one BatchTopK SAE per policy stream, LIBERO suite, and layer using closed-loop activation collections from 50 rollouts per task. OpenVLA examples are residual states from autoregressive action-token forwards. PG examples are residual states from the PaliGemma prefix-prefill pass. AE examples are residual states from action-expert denoising forwards over predicted action chunks.

Hook locations. All SAEs are trained and applied at post-block residual activations, hooked after each selected transformer block. The block differs by stream: for OpenVLA, language-model blocks during action-token decoding; for PG, PaliGemma prefix blocks during the prefix-prefill pass; for AE, action-expert blocks during denoising. PG layer 17 is excluded: its prefix hidden state is discarded after KV-cache construction, so a post-block edit there cannot reach the action expert. AE activations are indexed by policy query timestep, denoising step, and action-token offset, with offset j aligned to environment timestep $t + j$.

Hyperparameter	Value
OpenVLA layers	{0, 16, 24, 31}
PG layers	{0, 5, 11, 16}
AE layers	{0, 5, 11, 17}
OpenVLA SAE size	4096 $\rightarrow$ 32768
PG SAE size	2048 $\rightarrow$ 2048
AE SAE size	1024 $\rightarrow$ 1024
Active budget k	64
Learning rate	5×10^{-5} (OpenVLA) 1×10^{-4} (PG/AE)
Batch size	40,000
Optimizer	Adam
Training steps	4k OpenVLA; 10k PG/AE
Warmup steps	1,000
Decay start	80% of training
Activation normalization	Enabled
Training dtype	float32

(a) SAE training hyperparameters.

Hyperparameter	Value
AWE error budget η	0.05
Scoring window radius w	5
Visual weight λ_v	1.0
State weight λ_s	0.5
Temporal weight λ_p	0.4
Cosine threshold (agglomerative)	0.18
Cluster coverage threshold	0.5

(b) Clustering hyperparameters.

Table 5: Reproducibility hyperparameters. (a) SAE training (size denotes activation dimension $\rightarrow$ dictionary size). (b) Keyframe extraction and event clustering; the three weights apply to the descriptor concatenation in Section 3.3, where each component is normalized before weighting and the concatenated descriptor is ℓ_2 -normalized.

Compute resources. All experiments were run on an internal academic GPU cluster with NVIDIA A100 GPUs. SAE training takes a few GPU-hours per BatchTopK SAE on a single A100; each LIBERO rollout takes approximately 1–2 minutes of wall-clock time on a single GPU. Activation collection and intervention sweeps were parallelized across cluster nodes to obtain the per-experiment rollout budgets specified in Section 4.1.

B Clustering Hyperparameters

This appendix details the kinematic keyframe extraction (Section 3.2) and event clustering (Section 3.3) stages; the hyperparameter values are given in Table 5b.

Hyperparameter selection. We calibrated the descriptor weights $\lambda_v, \lambda_s, \lambda_p$ so that the recurring-cluster count per task stays on the order of the typical AWE waypoint count per rollout, keeping clusters aligned with the kinematic event scale of the underlying trajectories. Each component (visual embedding, robot state, temporal-progress scalar) is independently ℓ_2 -normalized before weighting, so the weights set the relative contribution of each component to the cosine distance. The relative ordering $\lambda_v > \lambda_s > \lambda_p$ reflects the intuition that visual scene context is the most discriminative signal, end-effector state breaks ties between visually similar moments, and temporal progress provides a weak ordering prior.

C VLM Cluster Annotation Prompt

This appendix gives the prompt template used by the VLM cluster annotation step (Section 3.3); we use gemini-3.1-pro-preview as the VLM. The template is filled per cluster with the task instruction, the cluster identifier, the episode coverage, and the number of clips and frames-per-clip drawn from the cluster’s keyframe bundles.

You are labeling a recurring event in a robot manipulation task.

You will be shown <N> image sequences from different rollout episodes of the same task. Each sequence contains <F> images in chronological order. All sequences were clustered automatically and are intended to depict the same recurring event type. Treat each sequence as one short clip; the model input is therefore a small batch of clips.

Task instruction: "<task description>"

Cluster id: <cluster id>

Episode coverage: <coverage fraction>

Your job is to assign one canonical phrase describing the shared event across the sequences. Focus on the common event across clips, not small differences between episodes. Prefer a dynamic event description (e.g., releasing, approaching) over a static state description. If the later frames make the event clearer, prioritize them when choosing the phrase and phase.

Choose exactly one phase label from the closed set:

pre_grasp, immobilization, contact, detach, post_grasp, transition.

Return JSON with exactly two keys:

```
{
  "phrase": <short canonical event phrase>,
  "phase": <one label from the set above>
}
```

The top-level JSON value must be a single object, not a list. Output exactly one short human-readable phrase. Do not include any keys other than "phrase" and "phase", and do not include markdown fences.

D Feature Ranking Score Definitions

This appendix gives the per-episode and per-cluster score formulas for the four ranking strategies described in Section 3.4.

Notation. We use the following symbols throughout.

- $\mathbf{z}_{\rho,t} \in \mathbb{R}^m$: SAE activation vector at rollout ρ and environment timestep t ; $\mathbf{z}_{\rho,t}[f]$ is its feature- f component and m the dictionary size.
- w : scoring window radius (offsets $\delta \in \{-w, \dots, w\}$).
- Event i has rollout ρ_i and timestep t_i . Its windowed activation is $z_{f,\delta}^{(i)} := \mathbf{z}_{\rho_i, t_i + \delta}[f]$, with window mean $\bar{z}_f^{(i)} := \frac{1}{2w+1} \sum_{\delta=-w}^w z_{f,\delta}^{(i)}$.
- $\mathcal{R}$: clusters retained from Section 3.3; n_r : number of events in cluster r ; $\mathcal{E}_r$: episodes contributing to cluster r ; $\mathcal{C}_{r,e}$: events from episode e assigned to cluster r .
- $\mathcal{Q} = \{q_{\text{pulse}}, q_{\text{up}}, q_{\text{down}}\}$: the three temporal templates of Eq. (3); each is mean-centered and ℓ_2 -normalized before projection, so the cross-template maximum in Eq. (4) compares templates on a common scale.
- K : number of top features selected per ranking as the intervention candidate set.

Each ranking assigns a score to every (cluster, feature) pair and then reduces it to a feature-level ranking by a strategy-specific aggregation.

Event-aligned ranking. For each event i , the per-event template- q score (Eq. 2) projects the mean-subtracted windowed activation onto template q ; subtracting the window mean makes the score depend on temporal shape rather than baseline activation level. We use three templates (Eq. 3). The cluster-level score (Eq. 4) averages per-event scores within each episode, takes the best-matching template, and weights episodes equally. The suite-level score (Eq. 5) averages the cluster scores uniformly over $\mathcal{R}$.

$$s_i^q(f) = \max\left(\sum_{\delta=-w}^w (z_{f,\delta}^{(i)} - \bar{z}_f^{(i)}) q(\delta), 0\right) \quad (2)$$

$$q_{\text{pulse}}(\delta) = 1 - \frac{|\delta|}{w+1}, \quad q_{\text{up}}(\delta) = \begin{cases} -1 & \delta < 0 \\ +1 & \delta \geq 0 \end{cases}, \quad q_{\text{down}} = -q_{\text{up}} \quad (3)$$

$$A_{r,f} = \frac{1}{|\mathcal{E}_r|} \sum_{e \in \mathcal{E}_r} \max_{q \in \mathcal{Q}} \frac{1}{|\mathcal{C}_{r,e}|} \sum_{i \in \mathcal{C}_{r,e}} s_i^q(f) \quad (4)$$

$$R_{\text{event}}(f) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} A_{r,f} \quad (5)$$

Window-mean ranking. With the same episode-balanced aggregation as the event-aligned cluster score, the window-mean cluster score (Eq. 6) replaces the template projection with the plain window-mean activation; the suite-level score (Eq. 7) weights clusters by event count.

$$B_{r,f}^{\text{window}} = \frac{1}{|\mathcal{E}_r|} \sum_{e \in \mathcal{E}_r} \frac{1}{|\mathcal{C}_{r,e}|} \sum_{i \in \mathcal{C}_{r,e}} \frac{1}{2w+1} \sum_{\delta=-w}^w z_{f,\delta}^{(i)} \quad (6)$$

$$R_{\text{window}}(f) = \frac{\sum_{r \in \mathcal{R}} n_r B_{r,f}^{\text{window}}}{\sum_{r \in \mathcal{R}} n_r} \quad (7)$$

Task-mean ranking. The suite-level score (Eq. 8) is the mean activation over all (rollout, timestep) pairs in the suite.

$$R_{\text{task}}(f) = \text{mean}_{(\rho,t)} \mathbf{z}_{\rho,t}[f] \quad (8)$$

Random-alive ranking. A uniform sample of K alive SAE features in the suite, excluding any feature already selected by the other three rankings.

E Real-Robot Setup

This appendix describes the hardware, sensing, deployment, and logging configuration for the real-robot study in Section 4.4.

Hardware, sensing, and deployment. The Mobile ALOHA platform [37] controls a single follower arm over a 13.5×30 in tabletop workspace (Figure 6). Two Intel RealSense cameras provide RGB observations at 640×480 and 30 Hz: a wrist-mounted egocentric view on the active arm, and a third-person view from roughly 60 in above the workspace. Policy inputs are joint-position state, images resized to 224×224 , and the natural-language prompt.

We deploy the LoRA-finetuned $\pi_{0.5}$ via the OpenPI server/client pattern: a websocket server hosts the policy, and the real-robot client handles sensing and action execution at 30 Hz. Each rollout runs for up to 300 control steps, with the client requesting a new action chunk every 50 steps (matching the chunk horizon $H = 50$). At episode start, the arm is interpolated to the first predicted action over 5 s; between episodes it returns to a captured home pose.

Per-step logs include task and episode identifiers, prompt, executed actions, gripper state, timestamps, and end-effector state (reconstructed via forward kinematics). For interpretability analysis, we record $\pi_{0.5}$ action-expert activations during inference and align them to environment timesteps via $t_{\text{env}} = t_{\text{chunk}} + r$ for $r \in \{0, \dots, H-1\}$.

Value-vector steering baseline.

Conditions. We compare five conditions, each evaluated on 10 rollouts with counterbalanced layouts (5 red-left, 5 red-right): *Random* (a fixed random-vector intervention with matched strength), *Steer Red* and *Steer Yellow* (manual value-vector steering with the prompt Approach the chips), and *Prompt Red* and *Prompt Yellow* (prompt-only baselines that prepend the target color, e.g., Approach the red chips).

Vector selection (manual lexical shortlist). We adapt the FFN value-vector projection setup of Häon et al. [15], but replace the cluster-construction step with a deterministic manual lexical shortlist tailored to the red/yellow chip task. Candidate steering vectors are defined as the rows of the LoRA-merged FFN output matrix from the vocabulary-aligned PaliGemma expert (expert 0). Concretely, for an FFN output weight W with LoRA factors A, B , we form $W_{\text{eff}} = W + \lambda AB$ using the checkpoint’s LoRA scaling, flatten the layer and neuron dimensions, and treat each row as one candidate vector v_i . We project each v_i into the token embedding basis via $v_i E^\top$ and keep the top-10 decoded tokens for lexical inspection.

Each candidate is scored by a rank-weighted lexical match over its top-10 decoded tokens. We use red-word seeds $\mathcal{V}_{\text{red}} = \{\text{red, crimson, scarlet, reddish}\}$ and yellow-word seeds $\mathcal{V}_{\text{yellow}} = \{\text{yellow, gold, golden, amber, yellowish}\}$. For concept $c \in \{\text{red, yellow}\}$, the score is $S_c(i) = \sum_{r=1}^{10} (11-r) \mathbf{1}[t_{i,r} \in \mathcal{V}_c]$. We rank candidates by the red–yellow score contrast, retaining the top six vectors per concept. A spatial-confound score is used only as a filter and tie-breaker. The intervention coefficient is fixed to 10.



Figure 6: Chip-approach setup with counterbalanced red/yellow piles; orange dashed arrow shows a perturbed trajectory.

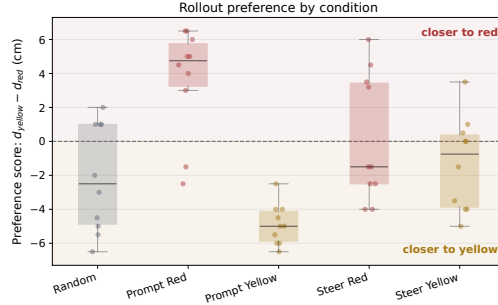


Figure 7: Value-vector steering preference score $\Delta d = d_{\text{yellow}} - d_{\text{red}}$: positive = closer to red, negative = closer to yellow. Prompt-only controls give the clearest color-conditioned effect; value-vector steering is noisier and overlaps the random-vector control.

Results. Figure 7 summarizes the rollout preference score $\Delta d = d_{\text{yellow}} - d_{\text{red}}$, where positive values indicate that the final trajectory came closer to the red chips and negative values indicate that it came closer to the yellow chips. Prompt-only controls produce the clearest color-conditioned behavior: the red prompt shifts rollouts toward positive Δd , while the yellow prompt shifts them toward negative Δd . In contrast, lexical-vector steering is substantially noisier. The random-vector control is not exactly zero-centered, which is expected in this small real-robot setting because rollout outcomes vary with chip layout and initial conditions even without semantic steering.

These results show that FFN value-vector projection can recover some color-associated directions, but the resulting interventions are weak and high-variance at the action level on this real-robot task. We read this not as a failure of Häon et al. [15], but as evidence that in VLAs the language and action subspaces remain partially entangled: lexical alignment in token-projection space is a useful but indirect proxy for behavioral control, since a feature with strong color semantics in vocabulary space

need not exert clean, isolable influence on the executed trajectory. This motivates our event-grounded SAE pipeline, which sidesteps the lexical proxy by selecting features from closed-loop behavioral events and validating them through action-level interventions.

F Event-Aligned Feature Score Visualization

Figure 8 visualizes per-event SAE feature scores as heatmaps; a high entry means the feature shows a consistent pulse or transition pattern around the event rather than merely high average activation.

The heatmaps are sparse and structured: a small subset of features forms high-scoring vertical bands across multiple VLM-labeled event rows. Since each row carries both a free-form phrase and a closed-set phase label, these bands suggest that some SAE features are not tied to a single event cluster but recur across semantically related manipulation phases such as approach, contact, transport, or release. Other high-scoring entries are more localized to individual rows, suggesting candidate event-specific associations. Note that this recurrence is only limited to OpenVLA.

We treat these heatmaps as a discovery aid; causal relevance is evaluated by the closed-loop zero-out experiments in Section 4.3. Figure 9 shows representative cluster-level keyframe bundles that feed the heatmaps.

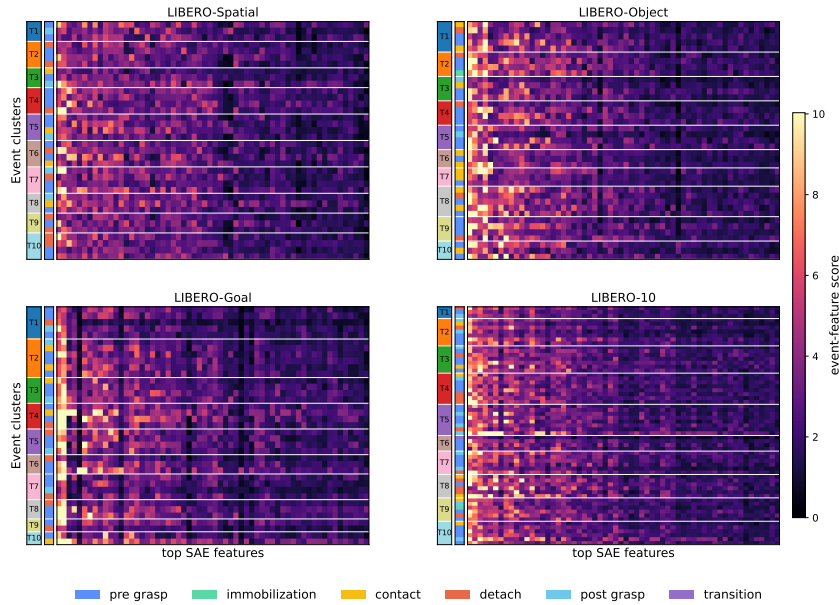


Figure 8: Event-aligned feature score heatmaps (OpenVLA, four LIBERO suites): rows are event clusters, columns are top layer-31 SAE features, cell color is the event-aligned score; side strips show task identity and phase labels.

G Zero-Out Failure Examples

Figure 10 shows representative zero-out failures on both architectures: the OpenVLA case disrupts target localization, grasping, and placement, while the $\pi_{0.5}$ action-expert case destabilizes the arm trajectory until rollout timeout. Figure 11 reports an additional gallery of paired baseline and zero-out rollout snapshots.

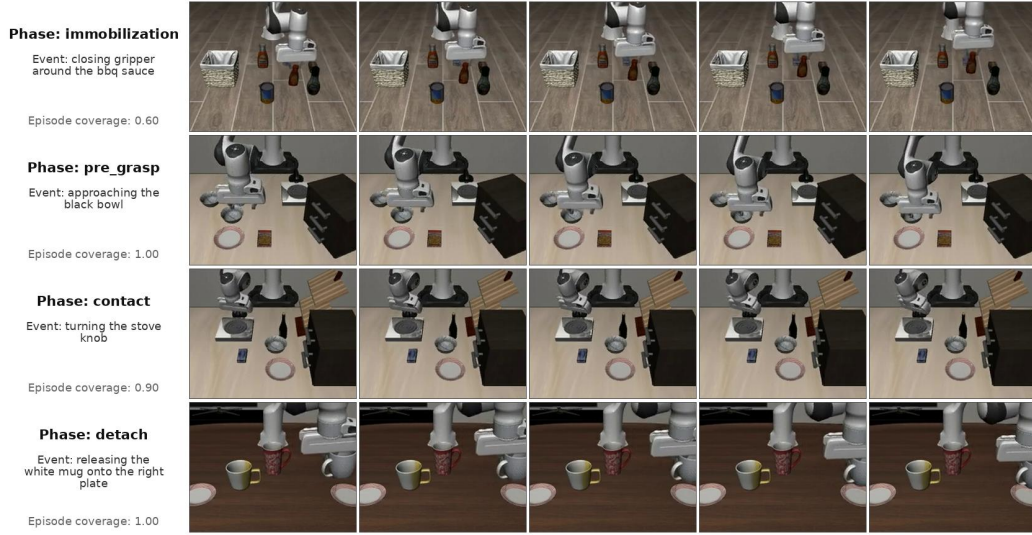
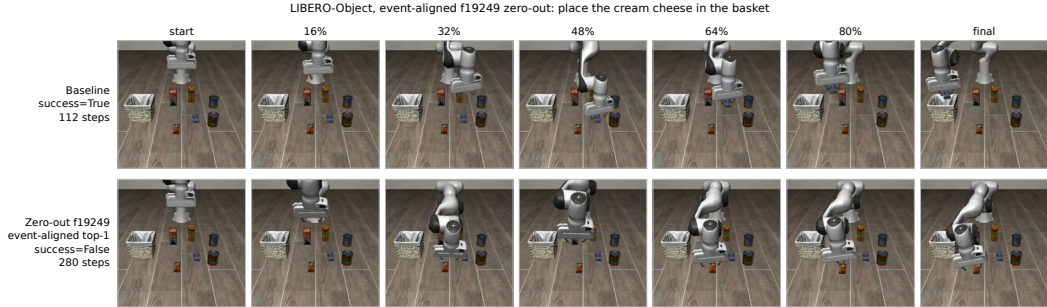
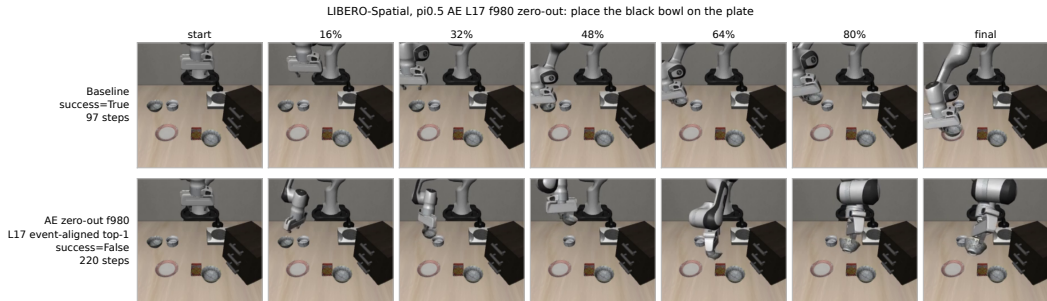


Figure 9: Four representative OpenVLA task-local event clusters; each row shows one keyframe bundle as a five-frame strip at offsets $\{-4, -2, 0, 2, 4\}$ around the keyframe timestep, with the cluster’s VLM-assigned phrase, phase label, and episode-coverage fraction.



(a) OpenVLA layer 31, event-aligned f_{19249} on LIBERO-Object place the cream cheese in the basket: the baseline succeeds while zero-out fails target localization, grasping, and placement.



(b) $\pi_{0.5}$ action-expert layer 17, event-aligned f_{980} on LIBERO-Spatial place the black bowl on the plate: the baseline completes in 97 steps while zero-out destabilizes the arm and times out at 220 steps.

Figure 10: This figure shows representative zero-out failures on both VLA architectures, evidencing that the SR drops in Table 3 can manifest as visible closed-loop degradation rather than only minor trajectory deviations.

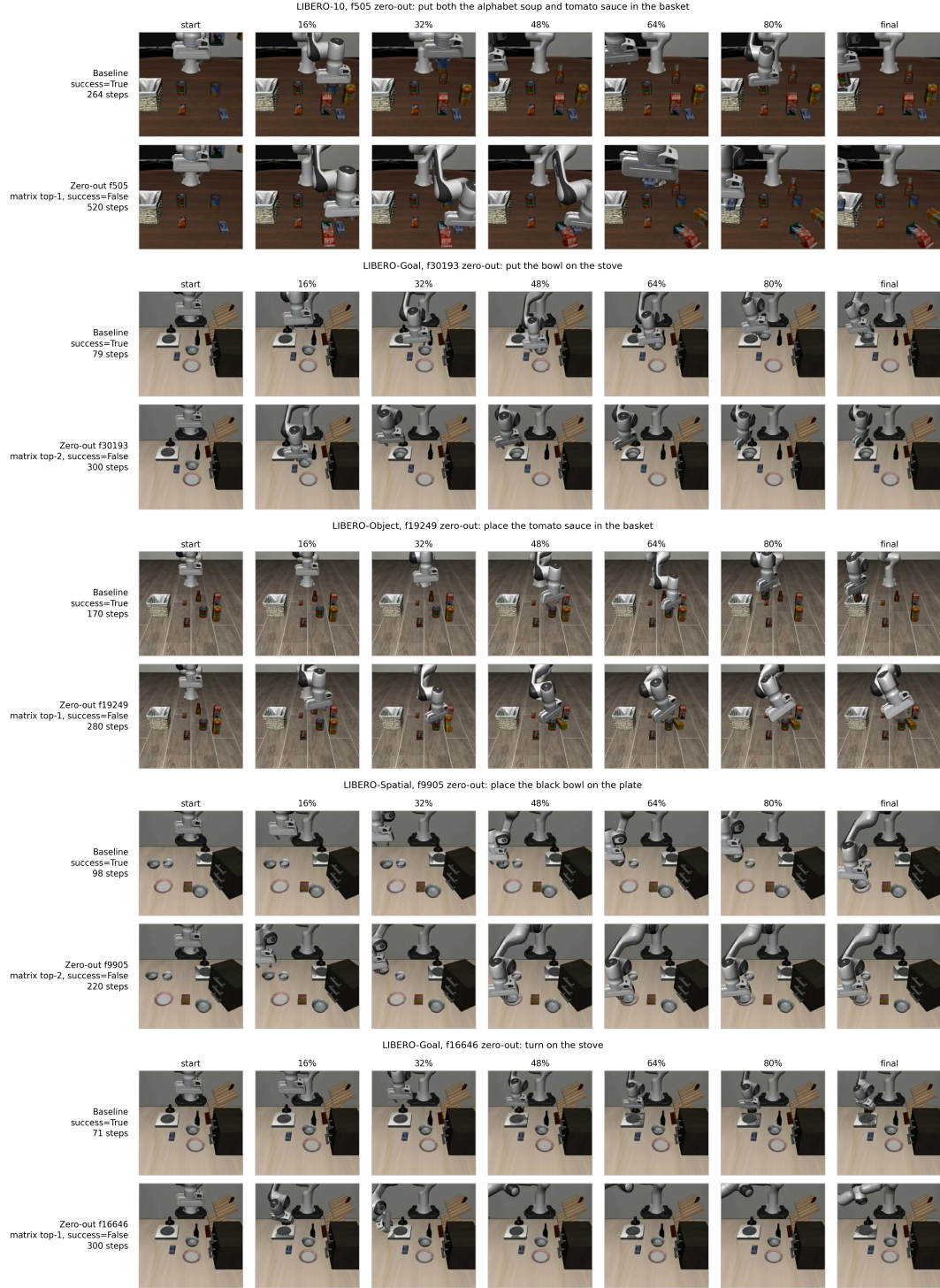


Figure 11: Additional paired rollout snapshots for suite-level zero-out; each row ablates one event-cluster-selected layer-31 SAE feature. Failures range from localized degradation (poor target localization, mistimed gripper) to larger control changes (e.g., LIBERO-Goal turn on the stove with feature f_{16646} induces markedly larger arm excursions).

H Joint Dropout Probe (PaliGemma Backbone)

Joint zero-out of 16 random active PG features (Table 6) shows two patterns.

(1) PG is more robust than AE, but only away from its first layer. At PG layers 5, 11, and 16, dropping 16 features at once gives small SR drops, far smaller than the near-total collapses that single AE features cause in Table 3. PG layer 0 is the exception: the same 16-feature dropout is severe there, and LIBERO-Goal falls to 8% SR. LIBERO-Goal uses one scene and object set for all ten tasks, so only the language instruction distinguishes them. In the layer-0 videos the policy still acts coherently but often performs a different LIBERO-Goal task, consistent with PG layer 0 carrying the instruction grounding.

(2) Within PG, the damage is largest at layer 0 and broadly shrinks with depth, with layer 16 essentially a no-op. This depth trend is consistent with the same amplification mechanism noted in Section 4.1: a perturbation at an earlier PG layer has more downstream layers to amplify it, while one near the final prefix block has almost none, so its effect stays small.

Suite	PG layer	Baseline SR (%)	Dropout SR (%)	Δ SR (pp)
LIBERO-Spatial	0	98.0	66.7 $\pm$ 6.8	−31.3
	5	98.0	86.7 $\pm$ 3.8	−11.3
	11	98.0	98.0 $\pm$ 1.6	+0.0
	16	97.8	100.0 $\pm$ 0.0	+2.2
LIBERO-Object	0	98.8	53.3 $\pm$ 4.7	−45.5
	5	98.8	88.7 $\pm$ 7.7	−10.1
	11	98.8	96.0 $\pm$ 1.6	−2.8
	16	98.8	100.0 $\pm$ 0.0	+1.2
LIBERO-Goal	0	97.8	8.0 $\pm$ 1.6	−89.8
	5	97.8	79.3 $\pm$ 2.5	−18.5
	11	97.8	96.7 $\pm$ 0.9	−1.1
	16	96.8	96.0 $\pm$ 1.6	−0.8
LIBERO-10	0	92.8	77.3 $\pm$ 2.5	−15.5
	5	92.8	84.0 $\pm$ 3.3	−8.8
	11	92.8	80.0 $\pm$ 7.1	−12.8
	16	93.2	94.7 $\pm$ 0.9	+1.5

Table 6: This table reports closed-loop success after randomly dropping 16 active PaliGemma SAE features. Values are percentages. Dropout SR reports the mean success rate $\pm$ standard deviation across three random seeds. Δ SR is computed as $SR_{\text{dropout}} - SR_{\text{baseline}}$ in percentage points. Negative values indicate lower success after intervention.

I Per-Feature Top- K Zero-Out Breakdown

This appendix shows per-feature Δ SR across top- K ranks for all three SAE streams. Figure 12 visualizes the OpenVLA layer-31 and $\pi_{0.5}$ PG/AE breakdowns; the following table reports the corresponding $\pi_{0.5}$ PG and AE per-feature drops in detail. All values are mean Δ SR = $SR_{\alpha=0} - SR_{\text{baseline}}$ in percentage points, averaged over the four LIBERO suites; negative values indicate lower success after intervention.

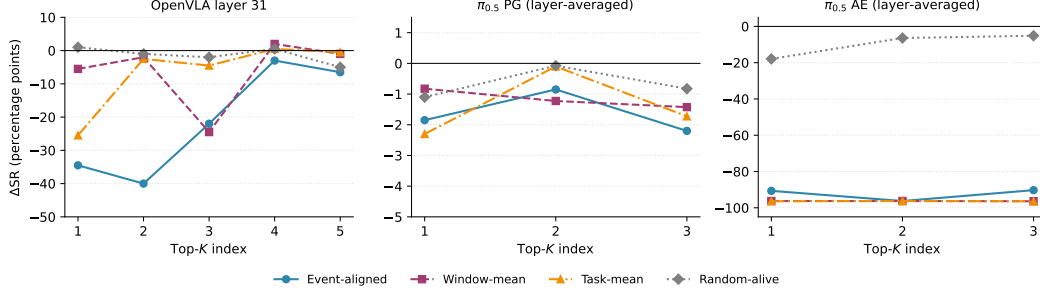


Figure 12: This figure shows per-feature ΔSR (percentage points) under zero-out for the three SAE streams. Left: OpenVLA layer 31 (top-1 to top-5). Middle: $\pi_{0.5}$ PG, layer-averaged across $\{0, 5, 11, 16\}$ (top-1 to top-3). Right: $\pi_{0.5}$ AE, layer-averaged across $\{0, 5, 11, 17\}$ (top-1 to top-3). Each panel uses an independent y-scale to preserve within-stream structure.

Layer	Ranking	top-1	top-2	top-3
0	Event-aligned	-1.4	-1.4	-3.9
	Window-mean	-1.4	-1.4	-1.4
	Task-mean	-1.4	+0.1	-1.4
	Random-alive	-0.4	+0.6	-2.4
5	Event-aligned	-2.4	+0.1	-0.4
	Window-mean	-0.4	-0.9	-1.9
	Task-mean	+0.2	-0.4	-0.4
	Random-alive	-0.9	+0.2	-0.9
11	Event-aligned	-3.4	-1.4	-3.4
	Window-mean	-1.9	-1.9	-2.8
	Task-mean	-4.9	+0.6	-4.4
	Random-alive	-1.9	-0.9	-0.4
16	Event-aligned	-0.2	-0.7	-1.1
	Window-mean	+0.4	-0.7	+0.4
	Task-mean	-3.1	-0.7	-0.7
	Random-alive	-1.2	-0.2	+0.4

(a) $\pi_{0.5}$ PG (PaliGemma backbone).

Layer	Ranking	top-1	top-2	top-3
0	Event-aligned	-96.4	-96.4	-96.4
	Window-mean	-96.4	-96.4	-96.4
	Task-mean	-96.4	-96.4	-96.4
	Random-alive	+0.6	-1.9	-0.9
5	Event-aligned	-94.4	-95.9	-96.4
	Window-mean	-96.4	-95.9	-96.4
	Task-mean	-96.4	-95.9	-96.4
	Random-alive	-23.9	-24.9	-21.4
11	Event-aligned	-96.4	-96.4	-96.4
	Window-mean	-95.9	-96.4	-96.4
	Task-mean	-96.4	-96.4	-96.4
	Random-alive	-24.9	-0.4	+1.1
17	Event-aligned	-75.4	-96.4	-71.9
	Window-mean	-96.4	-96.4	-96.4
	Task-mean	-96.4	-96.4	-96.4
	Random-alive	-23.4	+1.6	+0.6

(b) $\pi_{0.5}$ AE (action expert).

Table 7: Per-feature $\Delta\text{SR} = \text{SR}_{\alpha=0} - \text{SR}_{\text{baseline}}$ (percentage points) for $\pi_{0.5}$ zero-out interventions, broken down across the top-3 features per ranking and averaged over the four LIBERO suites. (a) PG drops stay within roughly 5 pp across all rows. (b) AE event-aligned, window-mean, and task-mean entries cluster near the policy-collapse floor (≈ -96), leaving the within-ranking ordering largely indistinguishable.

J Target/Off-Target Zero-Out Specificity Probe

To probe whether event-cluster-selected features are task-local in OpenVLA, we sample up to three event clusters per task and take each cluster’s highest-scoring SAE feature. We zero out that feature twice: once on the target task that selected the cluster, and once on one off-target task drawn at random from the same suite (5 rollout trials each, Table 8). A task-specific feature should produce a larger SR drop on the target task than on the off-target task; similar drops indicate shared manipulation roles.

We read this not as negative evidence against event-feature scoring, but as a sign that many event-selected features participate in manipulation roles shared across tasks within a suite (reach, grasp, transport, release) rather than functioning as task-private circuits.

Suite	Event clusters	Target SR	Off-target SR
LIBERO-Spatial	30	0.740	0.753
LIBERO-Object	30	0.607	0.560
LIBERO-Goal	28	0.693	0.729
LIBERO-10	30	0.527	0.487
All suites	118	0.641	0.631

Table 8: Target/off-target zero-out diagnostic for event-cluster-selected SAE features. We sample up to three event clusters per task; two LIBERO-Goal tasks have only two event clusters, so LIBERO-Goal contributes 28 clusters rather than 30. Each cluster is evaluated on its target task and on one off-target task from the same suite. Target and off-target SR are nearly identical on average.

K Decoder-Vector Steering Diagnostics

Decoder-vector steering adds a feature’s decoder vector to the hidden state, $x'_t = x_t + \alpha d_i$. At large α this update can dominate the native residual stream, so we measure the relative update magnitude $\rho = \|\alpha d_i\|_2 / \|x_t\|_2$ and the post-intervention alignment $c_{\text{after}} = \cos(x'_t, d_i)$ alongside SR. Using feature f_{24287} at OpenVLA layer 31 with $\alpha = 150$ (the steering scale used by Swann et al. [14]) on LIBERO-Object (10 tasks $\times$ 5 rollouts), we compare decoder-vector steering against a random unit vector matched to the same update norm (Table 9).

Intervention	SR	$\mathbb{E}[\rho]$	$\mathbb{E}[c_{\text{before}}]$	$\mathbb{E}[c_{\text{after}}]$	$\ x_t\ _2$	$\ x'\ _2$
Decoder vector d_i	0.00	1.54	0.012	0.840	97.6	180.0
Random unit vector r	0.52	1.54	0.008	0.839	97.6	179.6

Table 9: Perturbation-scale diagnostics for decoder-vector steering versus a random-vector control with matched update norm. $c_{\text{before}} = \cos(x_t, v)$ and $c_{\text{after}} = \cos(x'_t, v)$, where v is the injected direction (d_i for the decoder vector, r for the random control).



Figure 13: Qualitative comparison on LIBERO-Object pick up the cream cheese and place it in the basket (f_{24287} , layer 31, $\alpha = 150$). Top: baseline succeeds. Middle: decoder-vector steering drives the arm far off-task, then locks into that pose. Bottom: matched random-vector control produces erratic off-task motion.

The decoder direction collapses behavior far more than the random vector (0% vs 52% SR), confirming d_i is behaviorally relevant. But with $\rho > 1$, the injected update is larger than the native residual stream, so the edit may overwrite substantial parts of x_t rather than act as a clean feature edit. The two failure modes are qualitatively distinct (Figure 13): decoder steering drives the arm through a large off-task displacement and then slowly locks into that displaced pose (consistent with the high c_{after}), while the random control fails through erratic motion. The residual-preserving latent edit ($x' = x + \text{Dec}(z') - \text{Dec}(z)$, Section 3.5) avoids this overwrite issue by depending on the current sample’s code; we use it as the primary causal test.

L Sanity Checks

Success predictability of mean-pooled SAE codes in OpenVLA. As a static sanity check on top of the reconstruction results in Section 4.1, we ask whether mean-pooled SAE activations carry rollout-level outcome information. For each rollout ρ we mean-pool its per-timestep SAE activations $\mathbf{z}_{\rho,t} \in \mathbb{R}^m$ (Appendix D) into $\bar{\mathbf{z}}_{\rho} = (1/T) \sum_{t=1}^T \mathbf{z}_{\rho,t}$, and train an L_2 -regularized logistic-regression probe to predict the binary success label with 5-fold stratified cross-validation. Table 10 compares the SAE probe against raw hidden states, task-identity, and shuffled labels. The picture is consistent with reconstruction fidelity: SAE codes retain behaviorally-relevant signal but are not lossless. We use mean-pooled codes only for this static probe. Per-token processing is essential for action fidelity [18], so all SAE training and interventions in the main results use per-token activations.

Suite	SAE	Raw hidden	Task-id only	Shuffled labels
LIBERO-Spatial	0.794	0.976	0.550	0.498
LIBERO-Object	0.926	0.942	0.636	0.498
LIBERO-Goal	0.837	0.896	0.627	0.576
LIBERO-10	0.791	0.893	0.538	0.484

Table 10: Balanced accuracy of trajectory-level success-prediction probes (5-fold cross-validation).

Behavioral comparison of intervention hook sites. We compare two SAE hook sites on OpenVLA LIBERO-Spatial layer 31: `mlp_out` (after the MLP block) and `resid_post` (after the residual addition; used in the main paper). Both use one matched protocol: single-feature zero-out of each site’s own top-5 event-aligned features, with the same event clusters and the same 10-task five-trial sweep, so only the SAE and hook site differ (Figure 14). `mlp_out` has higher reconstruction fidelity (0.83 vs. 0.67 closed-loop success under the SAE reconstruction hook) but a weak causal effect: its top-5 zero-out leaves success at 0.752 ± 0.060 , close to the 0.80 no-intervention baseline. `resid_post` drops far below baseline at its strongest features (0.516 ± 0.272 ; standard deviations are over the five features, population). Because this paper studies interventions, the deciding criterion is whether single-feature edits produce a behavioral effect large enough to measure: `resid_post` does, while `mlp_out`’s effect stays within the noise of a 50-rollout estimate and the policy’s own run-to-run variation. We therefore use `resid_post` as the main hook site and treat `mlp_out` as incremental.

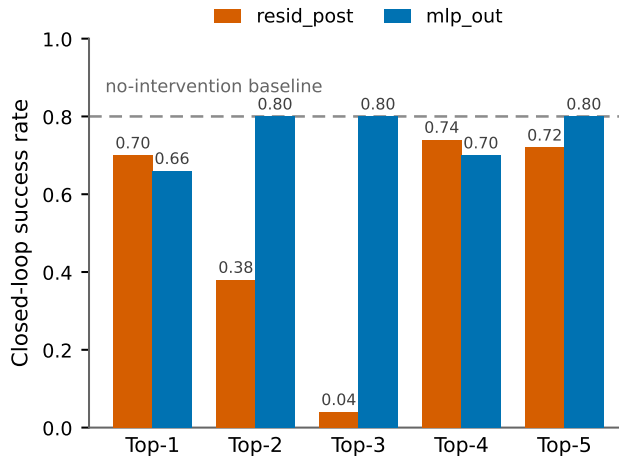


Figure 14: Per-rank single-feature zero-out closed-loop success rate for `resid_post` versus `mlp_out` at OpenVLA LIBERO-Spatial layer 31; the dashed line marks the no-intervention baseline. The matched protocol and the comparison are described in the text.